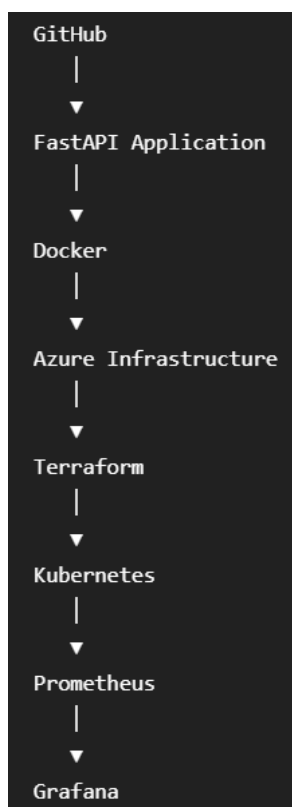


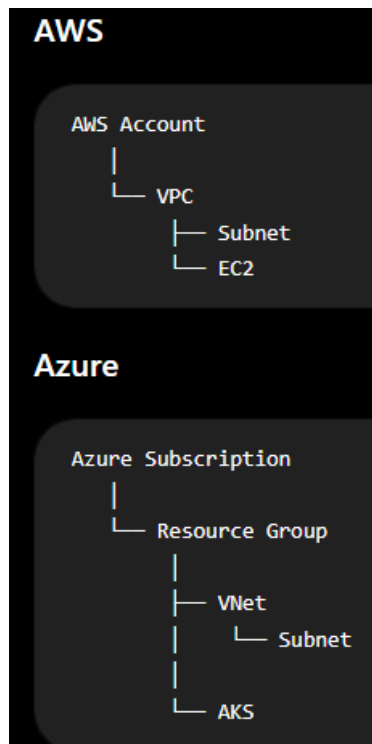
Target this phase 2: Azure – Terraform – Kubernetes

Notes: Because I can't log in to my AWS Console, I decided to switch from AWS to Azure.

AWS	Azure
Account	Subscription
VPC	VNet
Subnet	Subnet
EC2	Virtual Machine
EKS	AKS
Security Group	NSG
IAM	Microsoft Entra ID + RBAC
IAM Role	Azure RBAC / Managed Identity
CloudFormation	ARM/Bicep
AWS Region	Azure Region
Resource	Resource
Tags	Tags



Think of Azure as the main platform that contains all the resources for our project.



WARM-UP

1. Since I normally use AWS, this is my first time using Azure. I started by installing the Azure CLI (https://learn.microsoft.com/en-us/cli/azure/install-azure-cli-windows?view=azure-cli-latest&utm_source=chatgpt.com&pivots=msi)

After install it, check: **az --version**

```
PS C:\WINDOWS\system32> az --version
azure-cli                2.90.0

core                     2.90.0
telemetry                1.1.0

Dependencies:
msal                     1.36.0
azure-mgmt-resource     24.0.0

Python location 'C:\Program Files\Microsoft SDKs\Azure\CLI2\python.exe'
Config directory 'C:\Users\Valin\.azure'
Extensions directory 'C:\Users\Valin\.azure\cliextensions'

Python (Windows) 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)]

Legal docs and information: aka.ms/AzureCliLegal

Your CLI is up-to-date.
PS C:\WINDOWS\system32> |
```

2. Login Azure
az login

```
PS C:\WINDOWS\system32> az login
Select the account you want to log in with. For more information on login with Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

No      Subscription name      Subscription ID      Tenant
-----
[1] *   Azure subscription 1  00e7e36a-186f-4d39-a117-ab05fbd6b847  Default Directory

The default is marked with an *; the default tenant is 'Default Directory' and subscription is 'Azure subscription 1' (00e7e36a-186f-4d39-a117-ab05fbd6b847).

Select a subscription and tenant (Type a number or Enter for no changes): 00e7
Invalid selection.
Select a subscription and tenant (Type a number or Enter for no changes): 00e7e36a-186f-4d39-a117-ab05fbd6b847
Invalid selection.
Select a subscription and tenant (Type a number or Enter for no changes):

Tenant: Default Directory
Subscription: Azure subscription 1 (00e7e36a-186f-4d39-a117-ab05fbd6b847)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://go.microsoft.com/fwlink/?linkid=2271236
```

az account show

```
PS C:\WINDOWS\system32> az account show
{
  "environmentName": "AzureCloud",
  "homeTenantId": "62ab525c-f89b-49ac-94e1-257f270456d4",
  "id": "00e7e36a-186f-4d39-a117-ab05fbd6b847",
  "isDefault": true,
  "managedByTenants": [],
  "name": "Azure subscription 1",
  "state": "Enabled",
  "tenantDefaultDomain": "valinfernanda08gmail.onmicrosoft.com",
  "tenantDisplayName": "Default Directory",
  "tenantId": "62ab525c-f89b-49ac-94e1-257f270456d4",
  "user": {
    "name": "valinfernanda08@gmail.com",
    "type": "user"
  }
}
```

az account list --output table

```
PS C:\WINDOWS\system32> az account list --output table
Name      CloudName      SubscriptionId      TenantId      State      IsDefault
-----
Azure subscription 1  AzureCloud      00e7e36a-186f-4d39-a117-ab05fbd6b847  62ab525c-f89b-49ac-94e1-257f270456d4  Enabled  True
PS C:\WINDOWS\system32>
```

az account list-locations --query

"[?name=='westeurope'].{Name:name,DisplayName:displayName}" --output table

```
PS C:\WINDOWS\system32> az account list-locations --query "[?name=='westeurope'].{Name:name,DisplayName:displayName}" --output table
Name      DisplayName
-----
westeurope  West Europe
```

3. Make Resource Group
In AWS, we call it as VPC

```
Azure Subscription
└─ Resource Group
    └─ VNet
```

az group create `

--name rg-devops-platform `

--location westeurope

```
PS C:\WINDOWS\system32> az group create `
>> --name rg-devops-platform `
>> --location westeurope
{
  "id": "/subscriptions/00e7e36a-186f-4d39-a117-ab05fbd6b847/resourceGroups/rg-devops-platform",
  "location": "westeurope",
  "managedBy": null,
  "name": "rg-devops-platform",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
```

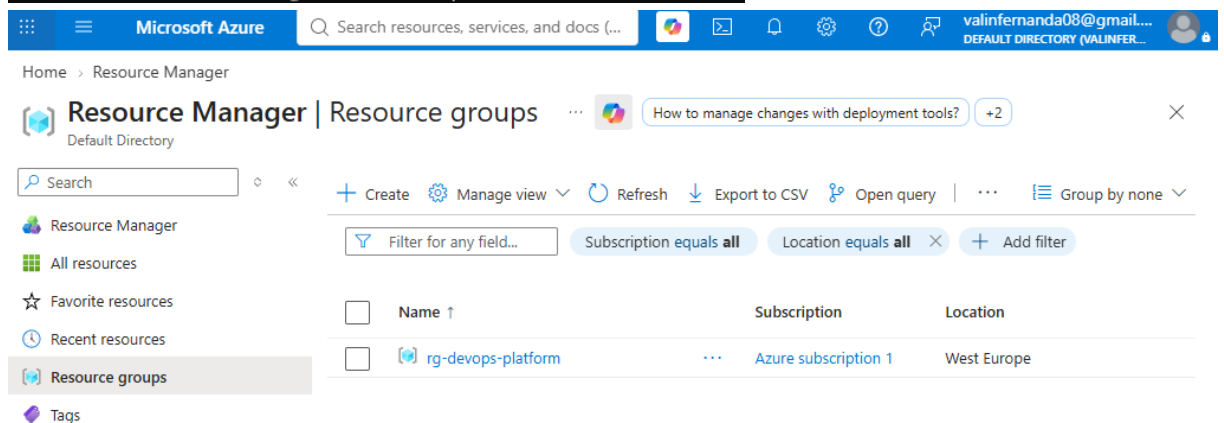
How to check Resource Group:

az group show `

--name rg-devops-platform `

--output table

```
PS C:\WINDOWS\system32> az group show `
>> --name rg-devops-platform `
>> --output table
Location      Name
-----
westeurope    rg-devops-platform
PS C:\WINDOWS\system32> |
```



Microsoft Azure

Search resources, services, and docs (...)

Home > Resource Manager

Resource Manager | Resource groups

Default Directory

Search

+ Create Manage view Refresh Export to CSV Open query ... Group by none

Filter for any field...

Subscription equals all Location equals all Add filter

Name ↑	Subscription	Location
rg-devops-platform	Azure subscription 1	West Europe

However, we will **delete the Resource Group we created manually and recreate it using Terraform.**

Why?

Because we want our portfolio to show:

Infrastructure → created with Terraform

az resource list --resource-group rg-devops-platform --output table

If it's **empty / has no resources**, it's safe to delete.

```
az group delete --name rg-devops-platform
```

```
PS C:\WINDOWS\system32> az resource list --resource-group rg-devops-platform --output table

PS C:\WINDOWS\system32> az group delete --name rg-devops-platform
Are you sure you want to perform this operation? (y/n): y
PS C:\WINDOWS\system32> |
```

Now we move on to the more important part of the portfolio: Terraform → Azure.

1. Check Terraform

```
terraform version
```

```
PS C:\WINDOWS\system32> terraform version
Terraform v1.13.3
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.16.2. You can update by downloading from https://developer.hashicorp.com/terraform/install
PS C:\WINDOWS\system32>
```

2. Move to folder project and make terraform folder

2a.

```
cd C:\Users\Valin\Downloads\cloud-native-devops-platform
```

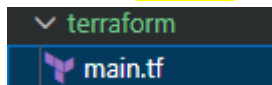
2b.

```
mkdir terraform
```

2c.

```
cd terraform
```

3. Make file `main.tf` in terraform folder



Enter this:

```
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

required_version = ">= 1.6.0"
}
```

```
provider "azurerm" {  
  features {}  
}
```

What are we actually doing?

Simply:

Terraform

↓

"I want to use Azure"

↓

AzureRM Provider

↓

Azure

azurerm = the bridge between Terraform and Azure.

4. Initialize Terraform

Still in terraform folder:

terraform init

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform init  
Initializing the backend...  
Initializing provider plugins...  
- Finding hashicorp/azurerm versions matching "~> 4.0"...  
- Installing hashicorp/azurerm v4.81.0...  
- Installed hashicorp/azurerm v4.81.0 (signed by HashiCorp)  
Terraform has created a lock file .terraform.lock.hcl to record the provider  
selections it made above. Include this file in your version control repository  
so that Terraform can guarantee to make the same selections by default when  
you run "terraform init" in the future.  
  
Terraform has been successfully initialized!
```

5. Make Resource Group

In folder

cloud-native-devops-platform/

└─ terraform/

Edit main.tf become:

```
terraform {  
  required_providers {  
    azurerm = {  
      source = "hashicorp/azurerm"  
      version = "~> 4.0"  
    }  
  }  
}
```

```

    required_version = ">= 1.6.0"
  }

  provider "azurerm" {
    features {}
  }

  resource "azurerm_resource_group" "main" {
    name     = "rg-devops-platform"
    location = "West Europe"

    tags = {
      project     = "cloud-native-devops-platform"
      environment = "dev"
      managed_by  = "terraform"
    }
  }
}

```



```

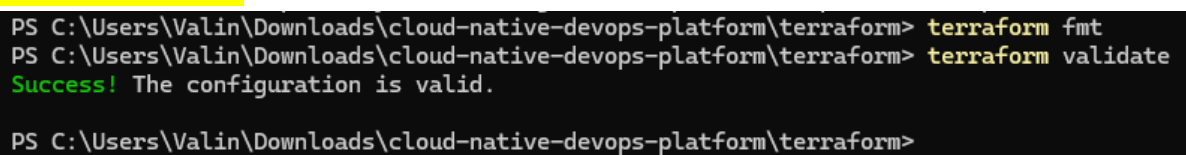
terraform > main.tf > resource "azurerm_resource_group" "main"
1 terraform {
2   required_providers {
3     azurerm = {
4       source = "hashicorp/azurerm"
5       version = "~> 4.0"
6     }
7   }
8
9   required_version = ">= 1.6.0"
10 }
11
12 provider "azurerm" {
13   features {}
14 }
15
16 resource "azurerm_resource_group" "main" {
17   name     = "rg-devops-platform"
18   location = "West Europe"
19
20   tags = {
21     project     = "cloud-native-devops-platform"
22     environment = "dev"
23     managed_by  = "terraform"
24   }
25 }

```

6. Check Terraform

terraform fmt

terraform validate



```

PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform fmt
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform validate
Success! The configuration is valid.

PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform>

```

7. See what will be made

Important concept in Terraform

terraform plan

↓

"This what I will do"

↓

terraform apply

↓

"OK, will do"

↓

Azure

Now, lets do terraform plan

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

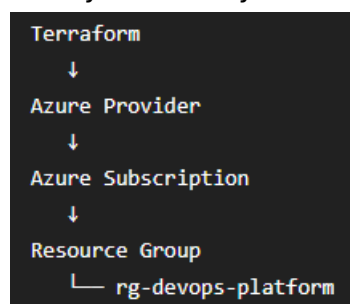
# azurerm_resource_group.main will be created
+ resource "azurerm_resource_group" "main" {
+   id           = (known after apply)
+   location     = "westeurope"
+   name         = "rg-devops-platform"
+   tags         = {
+     "environment" = "dev"
+     "managed_by"  = "terraform"
+     "project"     = "cloud-native-devops-platform"
+   }
}

Plan: 1 to add, 0 to change, 0 to destroy.
```

Its written Plan: 1 to add (that's what we want)

After we check and everything is fine, do Terraform apply

Finally we already have done:



8. Make VNet

AWS		Azure
VPC	→	VNet
Subnet	→	Subnet
Security Group	→	NSG

8a. Add VNet to main.tf

Under resource group, add:

```
resource "azurerm_virtual_network" "main" {  
  name          = "vnet-devops-platform"  
  location      = azurerm_resource_group.main.location  
  resource_group_name = azurerm_resource_group.main.name  
  
  address_space = ["10.0.0.0/16"]  
  
  tags = {  
    project = "cloud-native-devops-platform"  
    environment = "dev"  
    managed_by = "terraform"  
  }  
}
```

```
terraform > main.tf > ...  
1  terraform {  
2    required_providers {  
3      azurerm = {  
4        source = "hashicorp/azurerm"  
5        version = "~> 4.0"  
6      }  
7    }  
8  
9    required_version = ">= 1.6.0"  
10 }  
11  
12 provider "azurerm" {  
13   features {}  
14 }  
15  
16 resource "azurerm_resource_group" "main" {  
17   name     = "rg-devops-platform"  
18   location = "West Europe"  
19  
20   tags = {  
21     project     = "cloud-native-devops-platform"  
22     environment = "dev"  
23     managed_by  = "terraform"  
24   }  
25 }  
26  
27 resource "azurerm_virtual_network" "main" {  
28   name          = "vnet-devops-platform"  
29   location      = azurerm_resource_group.main.location  
30   resource_group_name = azurerm_resource_group.main.name  
31  
32   address_space = ["10.0.0.0/16"]  
33  
34   tags = {  
35     project     = "cloud-native-devops-platform"  
36     environment = "dev"  
37     managed_by  = "terraform"  
38   }  
39 }
```

8b. Format + validate

terraform fmt

terraform validate

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform fmt
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform validate
Success! The configuration is valid.
```

8c. Plan

terraform plan

terraform apply

Getting error:

```
azurerm_virtual_network.main: Creating...
Error: creating Virtual Network (Subscription: "00e7e36a-186f-4d39-a117-ab05fbd6b847"
Resource Group Name: "rg-devops-platform"
Virtual Network Name: "vnet-devops-platform"): performing CreateOrUpdate: unexpected status 403 (403 Forbidden) with error: RequestD
isallowedByAzure: Resource 'vnet-devops-platform' was disallowed by Azure: The selected region is currently not accepting new customer
s: https://aka.ms/locationineligible.

with azurerm_virtual_network.main,
on main.tf line 27, in resource "azurerm_virtual_network" "main":
27: resource "azurerm_virtual_network" "main" {
```

So I change the region to “Central US”

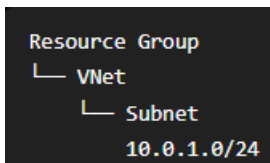
9. Make Subnet for AKS

In the main.tf, under resource Vnet, add:

```
resource "azurerm_subnet" "aks" {
  name                = "snet-aks"
  resource_group_name = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name

  address_prefixes = ["10.0.1.0/24"]
}
```

So now, our networking structure is:



VNet: 10.0.0.0/16

Subnet: 10.0.1.0/24

10.0.0.0/16

|

├── 10.0.1.0/24 ← AKS

├── 10.0.2.0/24 ← later can be used for another resources

└── ...

9a. Format and validate

```
terraform fmt
```

```
terraform validate
```

9b. Plan and Apply

```
terraform plan
```

```
terraform apply
```

Now that we have finished the **basic networking setup**:

Azure Subscription

└─ Resource Group

└─ VNet: 10.0.0.0/16

└─ Subnet: 10.0.1.0/24

10. Managed Identity

Small reminder:

In AWS, we call it an **IAM Role** (similar to **Managed Identity** in Azure) and an **IAM Policy** (similar to an **Azure RBAC Role**).

10a. Add Managed Identity

In main.tf, under the subnet, add this:

```
resource "azurerm_user_assigned_identity" "aks" {
    name          = "id-aks-devops-platform"
    resource_group_name = azurerm_resource_group.main.name
    location      = azurerm_resource_group.main.location
    tags = {
        project = "cloud-native-devops-platform"
        environment = "dev"
        managed_by = "terraform"
    }
}
```

```
resource "azurerm_user_assigned_identity" "aks" {
  name                       = "id-aks-devops-platform"
  resource_group_name       = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location

  tags = {
    project    = "cloud-native-devops-platform"
    environment = "dev"
    managed_by = "terraform"
  }
}
```

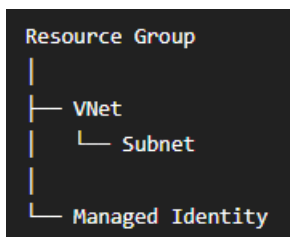
Why user_assigned?

There are two types of Managed Identities in Azure:

System-assigned → an identity is created and lives/dies with the resource.

User-assigned → an identity is a separate resource that can be used by other resources.

Now our Terraform structure:



10b. Format and validate

terraform fmt

terraform validate

terraform plan

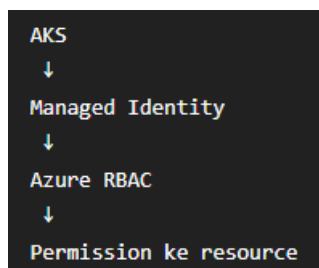
terraform apply

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform fmt
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform validate
Success! The configuration is valid.

PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform plan
```

Name	Type	Resource Group	Location
id-aks-devops-platform	Managed Identity	rg-devops-platform	Central US

11. Role Assignment



Now, we give that identity permission

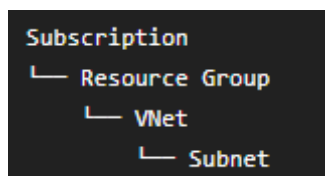
However, there's a slight change from the AWS analogy: role assignments must be attached to a specific scope (e.g., subscription, resource group, VNet, or subnet).

For our project, we'll assign the necessary identity permissions to the **VNet/subnet**.

Scope:

If a role is assigned to a Subscription, its scope is very broad.

If a role is assigned to a Subnet, its scope is much more specific.



11a. In main.tf, add this after Managed Identity:

```
resource "azurerm_role_assignment" "aks_network_contributor" {
```

```
scope = azurerm_virtual_network.main.id
```

```
role_definition_name = "Network Contributor"
```

```
principal_id = azurerm_user_assigned_identity.aks.principal_id
```

```
}
```

```
resource "azurerm_role_assignment" "aks_network_contributor" {  
  scope = azurerm_virtual_network.main.id  
  role_definition_name = "Network Contributor"  
  principal_id = azurerm_user_assigned_identity.aks.principal_id  
}
```

Read the concept from below:

WHO (Who is given permission?) → principal_id

WHAT (What role?) → role_definition_name

WHERE (where the role applies?) → scope

But what does "managing networking" mean?

For example, if the identity has a **Network Contributor**.

It can perform networking operations such as:



Short Notes:

Managed Identity = WHO

Network Contributor = WHAT CAN I DO

Scope = WHERE

11b. Format and validate

```
terraform fmt
```

```
terraform validate
```

```
terraform plan
```

```
terraform apply
```

12. AKS Cluster

In main.tf, under Role Assignment, add:

```
resource "azurerm_kubernetes_cluster" "main" {  
  name          = "aks-devops-platform"  
  location      = azurerm_resource_group.main.location  
  resource_group_name = azurerm_resource_group.main.name  
  dns_prefix    = "aks-devops-platform"
```

```
  default_node_pool {  
    name      = "system"  
    vm_size   = "Standard_D2s_v6"  
    node_count = 1  
    vnet_subnet_id = azurerm_subnet.aks.id  
  }  
}
```

```
  identity {  
    type = "UserAssigned"  
    identity_ids = [azurerm_user_assigned_identity.aks.id]  
  }  
}
```

```
  tags = {  
    project = "cloud-native-devops-platform"  
    environment = "dev"  
    managed_by = "terraform"  
  }  
}
```

```
resource "azurerm_kubernetes_cluster" "main" {
  name = "aks-devops-platform"
  location = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  dns_prefix = "aks-devops-platform"

  default_node_pool {
    name = "system"
    vm_size = "Standard_D2s_v6"
    node_count = 1
    vnet_subnet_id = azurerm_subnet.aks.id
  }

  identity {
    type = "UserAssigned"
    identity_ids = [azurerm_user_assigned_identity.aks.id]
  }

  tags = {
    project = "cloud-native-devops-platform"
    environment = "dev"
    managed_by = "terraform"
  }
}
```

resource "azurerm_kubernetes_cluster" "main" {

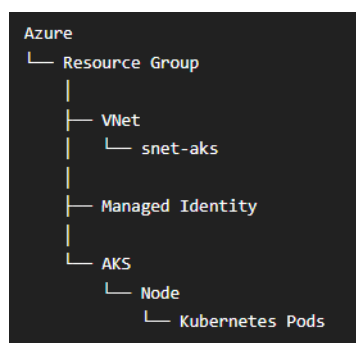
This means Terraform will create an Azure Kubernetes Service (AKS) cluster named main.

AKS Name

name = "aks-devops-platform"

This is the name of AKS that will appear in Azure.

This is what we just made:



⚠️ AKS + VM nodes are paid resources, so after testing is complete, we can **terraform destroy** them to avoid further costs.

terraform fmt

terraform validate

terraform plan

terraform apply

and...error

```
Error: creating Kubernetes Cluster (Subscription: "00e7e36a-186f-4d39-a117-ab05fbd6b847"
Resource Group Name: "rg-devops-platform"
Kubernetes Cluster Name: "aks-devops-platform"); performing CreateOrUpdate: unexpected status 400 (400 Bad Request) with response: {
  "code": "ServiceCidrOverlapExistingSubnetsCidr",
  "details": null,
  "message": "The specified service CIDR 10.0.0.0/16 is conflicted with an existing subnet CIDR 10.0.1.0/24. Please see https://aka.ms/aks/servicecidroverlap for how to f
ix the error.",
  "subcode": "",
  "target": "networkProfile.serviceCIDR"
}

with azure_rm_kubernetes_cluster.main,
on main.tf line 67, in resource "azure_rm_kubernetes_cluster" "main":
67: resource "azure_rm_kubernetes_cluster" "main" {
```

It means:

VNets: 10.0.0.0/16

Subnets: 10.0.1.0/24

AKS Service CIDR: 10.0.0.0/16 ← CLASH!

Why the conflict?

10.0.0.0/16 covers the entire range:

10.0.0.0

↓

10.0.255.255

While your subnet:

10.0.1.0/24

is within that range.

How to fix?

We simply assign a different AKS Service CIDR.

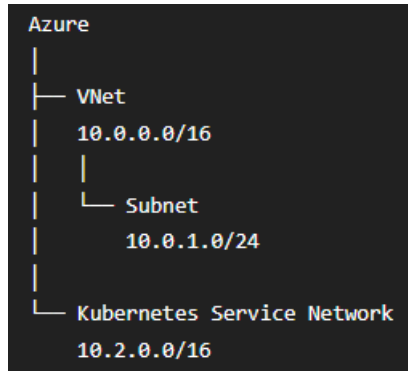
For example:

VNet:10.0.0.0/16

Subnet: 10.0.1.0/24

Kubernetes Service CIDR:10.2.0.0/16

So:



No overlap anymore.

Add network_profile

```
network_profile {
```

```
    network_plugin = "azure"
```

```
    service_cidr   = "10.2.0.0/16"
```

```
    dns_service_ip = "10.2.0.10"
```

```
}
```

```
default_node_pool {
  name           = "system"
  vm_size        = "Standard_D2s_v6"
  node_count     = 1
  vnet_subnet_id = azurerm_subnet.aks.id
}

identity {
  type           = "UserAssigned"
  identity_ids   = [azurerm_user_assigned_identity.aks.id]
}

network_profile {
  network_plugin = "azure"
  service_cidr   = "10.2.0.0/16"
  dns_service_ip = "10.2.0.10"
}

tags = {
  project      = "cloud-native-devops-platform"
  environment  = "dev"
  managed_by   = "terraform"
}
```

And now... error again **SkuNotAvailable**. SKU is actually a term for the type/kind of resource or product that Azure provides.

Standard_D2s_v6 is currently not available in location centralus

Solution:

Because Azure availability can change in real time, it's best to check the available SKUs for your subscription in Centralus, rather than trying multiple options.

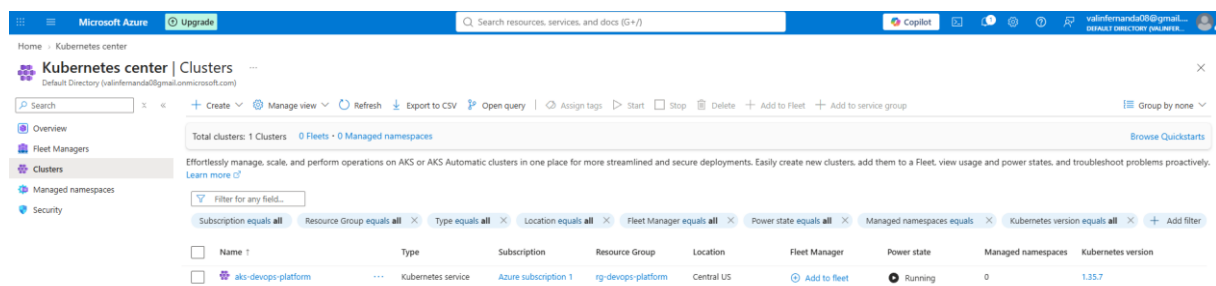
```
az vm list-skus --location centralus --resource-type virtualMachines --output table
```

This can result in a very long list of SKUs.

We can filter by relevant sizes. Try:

```
az vm list-skus --location centralus --resource-type virtualMachines --output json |  
Select-String "Standard_D|Standard_E|Standard_B"
```

So I change the node sizes into: Standard_b4ps_v2



Now, do some verification:

1. Check Terraform: terraform state list

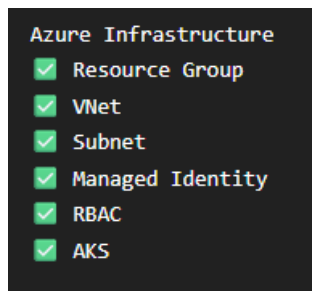
```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> terraform state list  
azurerm_kubernetes_cluster.main  
azurerm_resource_group.main  
azurerm_role_assignment.aks_network_contributor  
azurerm_subnet.aks  
azurerm_user_assigned_identity.aks  
azurerm_virtual_network.main  
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform>
```

2. Check AKS in Azure:

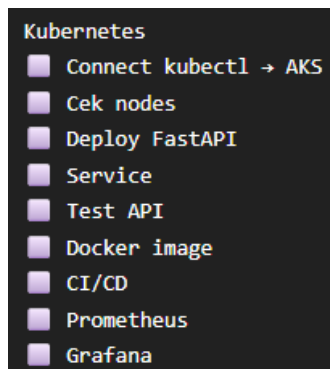
```
az aks show `  
--resource-group rg-devops-platform `  
--name aks-devops-platform `  
--output table
```

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> az aks show `  
>> --resource-group rg-devops-platform `  
>> --name aks-devops-platform `  
>> --output table  
Name Location ResourceGroup KubernetesVersion CurrentKubernetesVersion ProvisioningState Fqdn  
aks-devops-platform centralus rg-devops-platform 1.35 1.35.7 Succeeded aks-devops-platform-pl1bpg1l.hcp.centralus.azmk8s.io  
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform>
```

So, this is what we have made:



And next:



13. Connect kubectl to AKS

Connect your computer to the Kubernetes AKS cluster you created, so you can control AKS using the kubectl command.

```
az aks get-credentials `
```

```
--resource-group rg-devops-platform `
```

```
--name aks-devops-platform
```

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform> az aks get-credentials `
>> --resource-group rg-devops-platform `
>> --name aks-devops-platform
Merged "aks-devops-platform" as current context in C:\Users\Valin\.kube\config
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\terraform>
```

means kubectl is already connected to AKS.

Now, let us check whether the AKS node is really ready.

```
kubectl get nodes
```

If the node is Ready, it means our Kubernetes cluster is ready to receive application deployment.

Now, our project status:

Azure		
— Resource Group	✓	
— VNet	✓	
— Subnet	✓	
— Managed Identity	✓	
— Azure RBAC	✓	
— AKS		
— Node	✓	Ready

Terraform → Azure	✓	
kubectl → AKS	✓	
Kubernetes Node	✓	Ready

Cloud-Native DevOps Platform — Project Progress

✓ Completed

1. Project Setup

- Created GitHub repository
- Created project structure
- Created README.md
- Created .gitignore
- Created documentation structure

2. Application Development

- Built a Python FastAPI application
- Implemented Task Management API
- Implemented CRUD endpoints
 - GET /tasks
 - POST /tasks
 - GET /tasks/{id}
 - PUT /tasks/{id}
 - DELETE /tasks/{id}
- Added health endpoint

- Tested application locally
- Verified FastAPI Swagger UI

3. Docker

- Created Dockerfile
- Built Docker image
- Ran FastAPI inside Docker
- Tested port mapping (8000:8000)

4. Azure Infrastructure with Terraform

- Installed/configured Azure CLI
- Logged into Azure
- Selected Azure subscription
- Installed Terraform
- Configured AzureRM Terraform provider
- Initialized Terraform
- Created Azure Resource Group with Terraform
- Created Azure VNet with Terraform
- Created Azure Subnet with Terraform
- Configured VNet CIDR: 10.0.0.0/16
- Configured AKS subnet CIDR: 10.0.1.0/24
- Added Azure resource tags

5. Azure Identity & Permissions

- Created User-Assigned Managed Identity
- Configured Azure RBAC
- Assigned Network Contributor role
- Connected the identity to the AKS networking setup

6. Azure Kubernetes Service

- Created AKS cluster using Terraform
- Configured AKS node pool
- Connected AKS to the Azure subnet

- Configured AKS Managed Identity
 - Configured kubectl
 - Started the process of connecting kubectl to AKS
-

Current Step (STEP 2)

7. Kubernetes

Current position:

Azure

└─ AKS

└─ kubectl connection ← CURRENT

The next immediate check is:

kubectl get nodes

We want to see the node with:

STATUS

Ready

Next Steps

7. Kubernetes Deployment

- Create deployment.yaml
- Define FastAPI container
- Define replica count
- Configure container port
- Deploy application to AKS
- Check Pods
- Verify Pod status

8. Kubernetes Service

- Create service.yaml
- Expose FastAPI application

- Configure Service type
- Connect Service → Pods
- Test API through Kubernetes

Target:

Internet

↓

Kubernetes Service

↓

FastAPI Pods

↓

FastAPI Application

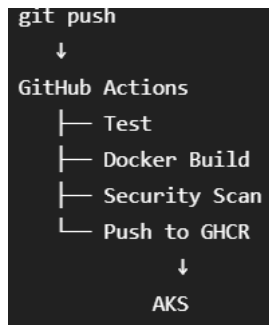
9. Container Registry

- Create/use GitHub Container Registry (GHCR)
- Push Docker image to GHCR
- Configure AKS to pull image
- Deploy GHCR image instead of local image

10. CI/CD — GitHub Actions

- Create GitHub Actions workflow
- Run automated tests
- Build Docker image
- Run security scan
- Push image to GHCR
- Deploy to AKS automatically

Target:



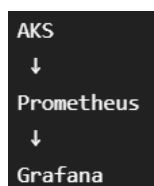
11. Container Security

- Add Trivy
- Scan Docker image
- Detect vulnerabilities
- Configure CI pipeline to fail on serious vulnerabilities

12. Monitoring

- Install Prometheus
- Configure Kubernetes metrics collection
- Install Grafana
- Connect Grafana → Prometheus
- Create dashboard
- Monitor:
 - CPU
 - Memory
 - Pod status
 - Request metrics
 - Kubernetes nodes

Target:



13. Documentation

- Complete architecture diagram

- Document Azure networking
- Document Terraform infrastructure
- Document Kubernetes deployment
- Document CI/CD pipeline
- Document monitoring
- Document security
- Update README
- Add screenshots/results

14. Final Portfolio Polish

- Clean Terraform code
- Clean Kubernetes manifests
- Clean GitHub Actions workflow
- Add architecture diagram
- Add setup instructions
- Add troubleshooting section
- Push final version to GitHub
- Make README recruiter-friendly

Final Architecture

By the end, the project should look like this:

